

INFO 0702 CM4 IA et virtualisation

Emilien Bourdy-Liébart



5 novembre 2025

Sommaire

- 1 Allocation de ressources
 - Migration de VM
 - Ressources dynamiques
 - Consommation d'énergie

Qu'est-ce que c'est ?

- Déplacement d'une VM d'un hôte source à un autre
- Migration à froid
- Migration à chaud
- Migration par Internet
- Problématiques :
 - ① Migration de la mémoire
 - ② Stockage partagé ← pas de migration
 - ③ Coûts
 - Downtime
 - Temps total de la migration
 - Nombre total de données transférées
 - Dégradation du service
 - Défauts de pages
 - Conflits d'adressage

Migration à froid

Mise de la machine hors tension pour copier l'état de la mémoire sur l'hôte cible.

Avantages

- ① Simple
- ② Aucune faute lors de la migration de la mémoire
- ③ La VM pourra redémarrer où elle en était

Inconvénients

- ① Arrêt des services
- ② Migration de la mémoire provoque un ralentissement sur le réseau

Migration à chaud I

Transfert d'une VM active *via* une connexion TCP.

Exemple (Brandford (1/2))

Utilisé par Xen, VMWare, KVM.

0 Pré-migration

VM active sur l'hôte source (hs), hyperviseur recherche l'hôte de destination (hd)

1 Réservation

Hyperviseur réserve un conteneur sur l'hd

2 Pré-copie itérative

Copie des pages mémoires sur l'hd en TCP

Si une page est modifiée sur l'hs, marquée comme sale

Étape répétée uniquement sur les pages modifiées

Arrêt si un nombre max d'itération est atteint ou si les pages salies sont toujours les mêmes

Migration à chaud II

Exemple (Brandford (2/2))

③ Arrêt et copie

Arrêt de la VM sur l'hs

Dernières pages sales copiées

Seule étape où la VM n'est plus accessible

→ *downtime* à minimiser

④ Validation

L'hd indique à l'hs qu'il a réceptionné la VM

L'hs envoie un accusé de réception et libère les ressources bloquées par la VM

⑤ Activation

La VM est activée sur l'hd et la table de routage du switch est mise à jour

Migration par Internet

Transfert de l'environnement d'exécution des VM d'un *cloud* à un autre.

- Chaque VM localisée dans un *cloud* a un état réseau (adresse IP, ports ouverts, connexions ouvertes etc.)
- Si migration à chaud, adresse IP publique de la VM doit resté la même durant la migration
- Solutions possibles
 - ① Utiliser un protocole IP mobile
 - ② Utilisation d'un DNS. Les VM gardent leurs noms canoniques, la nouvelle adresse IP est enregistrée avec le nom du serveur. Problématique dans la gestion du changement d'IP au niveau applicatif

Migration de la mémoire I

Pré-copie

- ① Phase d'échauffement (étape 2 de Brandford)
- ② Phase d'arrêt et copie (étape 3 de Brandford)

Migration de la mémoire II

Post-copie

- ❶ Arrêt de la VM sur l'hs
- ❷ Copie de l'état du processeur
- ❸ Redémarrage de la VM sur l'hd
- ❹ Copie des pages mémoires :
 - ❶ *Demand-paging* : demande par l'hd des pages manquantes à l'hs → ralentissement du système lié à la lenteur du réseau
 - ❷ *Active-push* : l'hs envoie les pages mémoires fautives
 - ❸ *Pre-paging* : l'hs prédit la prochaine page fautive sur l'hd
 - ❹ *Dynamic self ballooning* : *balloon driver* (bd) sur le *guest*.
Le bd peut réclamer des pages à l'OS et les donner à l'hyperviseur (*inflating*), ou demander des pages à l'hyperviseur et les donner à l'OS (*deflating*).

Migration de la mémoire III

Exemple (*Ballooning* de Xen)

Lors de l'initialisation d'une nouvelle VM, si l'hyperviseur n'a pas réservé assez de mémoire pour la nouvelle VM, Xen vole de la mémoire non utilisée d'autres VM (*inflate*) pour le nouveau *guest* et leur rend quand il n'en a plus besoin (*deflate*).

Post-copie hybride

- 1 Une seule étape de pré-copie
- 2 Arrêt de la VM sur l'hs
- 3 Copie de l'état du processeur et des pages modifiées
- 4 Démarrage de la VM sur l'hd
- 5 Post-copie

Optimisation de la migration par l'IA I

- ❶ Problématique : réduire la consommation énergétique pendant la migration tout en garantissant la performance des VM pendant le transfert.
- ❷ Entraînement du modèle de *machine learning* sur quelques caractéristiques : CPU, mémoire, stockage, réseau, CPU restant sur la PM, mémoire restante sur la PM, énergie consommée par la PM
- ❸ Sélection du modèle : Q-Learning, Deep RL, Deep Q-Networks etc.

Optimisation de la migration par l'IA II

Exemple (Articles de recherches associés)

- 1 Machine learning approach for live migration cost prediction in vmware environments
- 2 A multitime-steps-ahead prediction approach for scheduling live migration in cloud data centers
- 3 A Machine Learning Approach to Live Migration Modeling
- 4 A Hybrid Method for Short-Term Host Utilization Prediction in Cloud Computing
- 5 Machine Learning Based Statistical Prediction Model for Improving Performance of Live Virtual Machine Migration

Machine learning approach for live migration cost prediction in vmware environments

- Propose une approche ML pour VMWare qui prédit :
 - ① le temps de la migration à chaud
 - ② l'*overhead* réseau
 - ③ la consommation énergétique↳ en fonction de la taille mémoire de la VM
- Implémenté par script PowerCLI
 - Collection des données utilisées pendant la phase d'entraînement
 - 12 dernières heures
 - ⇒ taille mémoire de la VM, durée de la migration, débit réseau, pic de consommation énergétique
- Phase d'entraînement nourri par les migrations à chaud du DC

Modélisations I

Coût de la migration à chaud

- Débit réseau en fonction de la mémoire active :

$$R_s = \alpha e^{V_{mem}} + \beta \quad (1)$$

- R_s débit de l'hs en kBps
- V_{mem} mémoire active sur l'hs en kB quand la migration doit commencer
- α et β des constantes
- Si $V_{mem} = 0$, $R_s = \beta \Rightarrow$ CPU + contenu des *buffers* à migrer

Modélisations II

Durée de la migration

- Relation linéaire entre la durée de la migration et la taille de la mémoire selon le taux de transmission

$$T_{mig} = a \times \frac{V_{mem}}{R_s} + b \quad (2)$$

- T_{mig} durée de la migration en s
- a et b des constantes
- Si $V_{mem} = 0$, $T_{mig} = b \Rightarrow$ CPU à migrer

Modélisations III

Pic de consommation énergétique

- Relation linéaire avec le taux de transmission

$$P_{mig} = \frac{dE_{mig}}{dt} = c \frac{dV_{mig}}{dt} = cR_s \quad (3)$$

- P_{mig} le pic de consommation énergétique en Watt
- c une constante
- Si pas de donnée à transmettre, $P_{mig} = 0$

Modélisations IV

Prédictions

- α , β , a , b et c des constantes $\rightarrow$ pas de prédiction possible
- $\hookrightarrow$ Utilisation du ML pour les déterminer
- Régression sur les équations (1) à (3) par substitutions, puis *Root Mean Square Error*

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (d_i - f_i)^2} \quad (4)$$

- N le nombre de *samples* des 12 dernières heures
- d_i performance mesurée
- f_i valeur du modèle de régression
- Script lancé tant que $RMSE > 10\%$ sur les 12 heures

A multitime-steps-ahead prediction approach for scheduling live migration in cloud data centers

- Étude qui applique plusieurs algorithmes de prédictions pour optimiser les temps de migration et réduire les violations de niveau de service (SLAV) des DC
- Implémentation d'un *Recurrent Neural Network* (RNN) pour prédire les débits réseau et l'utilisation CPU
 - 2 entrées
 - ① Utilisation CPU/débit en cours
 - ② Utilisation CPU/débit précédente
 - 3 neurones dans la couche cachée
 - 1 sortie (la prédiction de l'utilisation du CPU/débit)
 - Chaque NN entraîné > 10 000 évaluations
 - Expérimentation > 10 fois

A Machine Learning Approach to Live Migration Modeling

- Propose une approche ML pour prédire 6 métriques
 - 1 Durée totale de la migration
 - 2 Volume total de donnée transféré
 - 3 *Downtime*
 - 4 Utilisation CPU sur l'hs
 - 5 Utilisation mémoire de l'hs
- Prédiction des 6 paramètres sur 5 algorithmes
 - 1 Pré-copie
 - 2 CPU *throttling*
 - 3 Delta compression
 - 4 Data compression
 - 5 Post-copie
- Choix de l'algorithme de migration dépend de la politique choisie (8 dans le papier)

Choix de l'algorithme de migration

- ❶ Éliminer les algorithmes qui vont SLAV
- ❷ Si plusieurs algorithmes restent, choix du meilleur
- ❸ Si aucun algorithme reste, celui qui minimise le SLAV
 - Somme des erreurs relatives
 - Exemple, modèle prédit 3,1 s de *downtime*, contrainte = 3 s,
erreur = $\left| \left(1 - \left(\frac{3.1}{3} \right) \right) \right| = 0.03$

A Hybrid Method for Short-Term Host Utilization Prediction in Cloud Computing

- Hybridation entre
 - ① Prédiction basé sur l'utilisation de l'hs sur le court terme sur un ensemble de décomposition en mode empirique (EEMD)
 - ② Moyenne mobile autorégressive intégrée (ARIMA)
- Ajout d'une phase de tests (RT)

⇒ EEMD-RT-ARIMA

Ensemble Empirical Mode Decomposition (EEMD) I

- EMD est une méthode qui décompose un signal en plusieurs fonctions de mode intrinsèques (IMF) et une tendance R

- ① Création d'une fonction

$$f(t) = x(t) \quad (5)$$

où $x(t)$ est la donnée originale

- ② Trouver les maximum et minimum locaux de $f_i(t)$ où i est un nombre de boucle commençant par 1. Interpolation entre les maximum et minimum locaux pour obtenir les enveloppes supérieures et inférieures. Calculer $m_i(t)$ la moyenne de ces enveloppes
- ③ Calculer $h_i(t) = f_i(t) - m_i(t)$
- ④ Vérifier que $h_i(t)$ vérifie les conditions des IMF :
 - ① Le nombre aux extrêmes ou au niveau du zéro doivent être les mêmes ou avoir une différence < 1

Ensemble Empirical Mode Decomposition (EEMD) II

- ② La moyenne des enveloppes = 0
 - Si non respect, faire $f_{i+1} = h_i(t)$ et refaire les étapes 2 et 3
 - Sinon calculer la tendance R avec $r_1(t) = x(t) - p_1(t)$ où $p_1(t) = h_i(t)$ qui satisfait l'IMF
- ⑤ Répéter 1-4 avec $r_1(t)$ la nouvelle donnée jusqu'à ce R soit une fonction monotone.
- $x(t)$ est décomposé en n IMF et R comme

$$x(t) = \sum_{i=1}^n p_i(t) + r(t) \quad (6)$$

- EEMD exécute EMD k fois en ajoutant du bruit blanc au signal à chaque étape

AutoRegressive Integrated Moving Average (ARIMA) I

- Modèle ARMA (autorégressif et moyenne mobile)
- Outil pour comprendre et prédire les valeurs futures d'une série
- 2 parties
 - ① autorégressive (AR)
 - ② moyenne mobile (MA)
- ARMA(p, q) où p est l'ordre de la partie AR et q l'ordre de la partie MA
- AR(p) s'écrit

$$X_t = c + \sum_{i=1}^p \varphi_i X_{t-1} + \varepsilon_t \quad (7)$$

Avec φ_i les paramètres du modèle, c une constante et ε_t un bruit blanc

AutoRegressive Integrated Moving Average (ARIMA) II

- $MA(q)$ s'écrit

$$X_t = \mu + \varepsilon_t + \sum_{i=1}^q \theta_i \varepsilon_{t-i} \quad (8)$$

Avec θ_i les paramètres du modèle, μ l'espérance de X_t et ε_t les termes d'erreur

- $ARMA(p, q)$ s'écrit

$$X_t = \varepsilon_t + \sum_{i=1}^p \varphi_i X_{t-i} + \sum_{i=1}^q \theta_i \varepsilon_{t-i} \quad (9)$$

- $ARIMA(p, d, q)$ dérive d fois X_t avant de le modéliser comme un $ARMA(p, q)$

Machine Learning Based Statistical Prediction Model for Improving Performance of Live Virtual Machine Migration

- Amélioration de la pré-copie de Xen
- Prédiction de pages sales
- Évaluation avec deux modèles
 - 1 Avec ARIMA
 - 2 Avec SVR (Support Vector Regression)

Résumé des articles I

Article	Technique	Bénéfices	Limitations
1	<i>Supervised learning</i>	Très efficace pour les migrations à chaud VMWare	Dégradation du service et coûts élevés
2	<i>Recurrent Neural Networks</i>	RNN efficace pour les prédictions CPU et réseau du <i>host</i>	La décision relative aux migrations doit être améliorée en ce qui concerne les paramètres de coût
3	<i>Machine learning model</i>	Bon taux de prédiction sur les environnements hétérogènes	Uniquement sur des scénarios simples

Résumé des articles II

Article	Technique	Bénéfices	Limitations
4	EEMD-RT-ARIMA	Méthode économique pour l'hôte	Gestion difficile entre ressources aléatoires et ressources à la demande
5	Modèles ARIMA et SVR	Bonne gestion des <i>workload</i> et de la migration à chaud	Difficile de passer d'un modèle de prédiction à un autre

Table 1 – Résumé des articles

Allocation des ressources dynamiques

- ↗ des serveurs $\implies$ ↗ volume de données imprévisibles et complication de leurs traitements
- Arrivée du *fog computing* (FC) $\longrightarrow$ problématique d'allocation des ressources au niveau des serveurs intermédiaires
- Problématique principale : trouver un bon algorithme d'ordonnancement en fonction des *workloads*

Optimisation de la gestion des ressources par l'IA I

Exemple (Articles de recherches associés (1/2))

- ➊ Resource scheduling for offline cloud computing using deep reinforcement learning
- ➋ Workflow scheduling using Neural Networks and Reinforcement Learning
- ➌ Multi-Objective Tasks Scheduling Algorithm for Cloud Computing Throughput Optimization

Optimisation de la gestion des ressources par l'IA II

Exemple (Articles de recherches associés (2/2))

- ④ A Task Scheduling Algorithm Based on Classification Mining in Fog Computing Environment
- ⑤ Task Scheduling in the Cloud Using Machine Learning Classification
- ⑥ An enhanced deadline constraint based task scheduling mechanism for cloud environment

Resource Scheduling for Offline Cloud Computing Using Deep Reinforcement Learning

- *Deep Reinforcement Learning* pour l'ordonnancement des ressources
- Extension de DeepRM et DeepRM2
- Uniquement CPU et mémoire
- Ajout
 - ➊ stockage
 - ➋ profile des *jobs*
- Objectifs de leur algorithme
 - ➊ Assigner les *jobs* aux VM appropriées pour minimiser la durée d'exécution
 - ➋ Satisfaire les *deadlines*
 - ➌ Minimiser les *slowdowns* des *jobs*

Workflow scheduling using Neural Networks and Reinforcement Learning

- Vision de l'ordonnancement des tâches comme un graphe orienté acyclique
 - Nœuds $\rightarrow$ tâches
 - Liens $\rightarrow$ dépendances entre les nœuds
- Application d'un ANN et du RL dessus

Multi-Objective Tasks Scheduling Algorithm for Cloud Computing Throughput Optimization I

- Problématique : les DC ne sont pas tous les mêmes, comment optimiser l'ordonnancement entre les DC ?
 - Tâches avec priorité élevée toujours exécutés
 - Tâches avec priorité faible moins souvent exécutés ou avec moins de CPU disponible
 - First Come First Serve et Shortest Job First fonctionnent très bien sauf dans le pire cas
 - Proposition :
 - Assigner les QoS pour les tâches et les VM
 - Tâches avec QoS élevé = valeur petite
 - Tâches avec QoS bas = valeur grande
- ⇒ QoS bas = grande priorité et QoS haut = faible priorité

Multi-Objective Tasks Scheduling Algorithm for Cloud Computing Throughput Optimization II

- VM avec MIPS élevé = QoS élevé
- VM avec MIPS bas = QoS bas
- Tri non dominant

$$\text{Minimize } f(S_k) = S_k | \forall j \exists i, f(S_i) \leq f(S_j) \quad (10)$$

$$\text{Minimize } f(Q_k) = Q_k | \forall j \exists i, f(Q_i) \leq f(Q_j) \quad (11)$$

$S, Q \in T(ID, Q, S)$ Avec S la taille de la tâche, Q la valeur du QoS, ID l'identifiant de la tâche, t la tâche

Multi-Objective Tasks Scheduling Algorithm for Cloud Computing Throughput Optimization III

Algorithm Multi-objective task scheduling algorithm (1/3)

1. Donner la liste des VM, des VM créés avec succès au DC et la liste des tâches au Broker

2. Créer une liste des tâches reçues

3. Créer une liste des VM reçues

4. Tri non dominant de la liste des tâches :

$i \leftarrow 0$

Crée une liste de non dominance vide

liste de dominance $\leftarrow$ liste de tâches

Mettre $task_i$ dans la liste non dominante

for $i \leftarrow 1$ à la taille de la liste des tâches **do**

for $j \leftarrow 0$ à la taille de la liste non dominante **do**

Multi-Objective Tasks Scheduling Algorithm for Cloud Computing Throughput Optimization IV

Algorithm Multi-objective task scheduling algorithm (2/3)

if $task_j$ domine $task_i$ **then**

Mettre $task_j$ dans l'ensemble non dominant

else if $task_i$ domine $task_j$ **then**

Mettre $task_i$ dans l'ensemble non dominant

else

Mettre $task_i$ et $task_j$ dans l'ensemble non dominant

end if

end for

end for

5. Trier la liste des tâches selon l'ensemble des tâches non dominantes

6. Trier la liste des VM reçues dans l'ordre décroissant

Multi-Objective Tasks Scheduling Algorithm for Cloud Computing Throughput Optimization V

Algorithm Multi-objective task scheduling algorithm (3/3)

7. $j \leftarrow 0$

for $i \leftarrow 0$ à la taille de la liste des tâches **do**

if $j \geq 0$ **then**

 Lier $task_i$ à la VM_j

$j++$

if $j == \text{nombre de VM}$ **then**

$j = 0$

end if

end if

end for

A Task Scheduling Algorithm Based on Classification Mining in Fog Computing Environment

- Ordonnancement des tâches entre les nœuds d'un *fog-computing*
- Les nœuds ont des puissances de calcul différentes
- Besoin de les ordonner pour que le temps d'exécution soit le plus petit
- Utilisation d'une *classification mining*
- Idée de l'algorithme
 - Ordonner les tâches dans table relationnelle selon leur priorité
 - Mettre une grande durée d'achèvement dans la table
 - Sélectionner le nœud avec la plus petite durée d'achèvement
 - Boucler sur le reste des tâches

Task Scheduling in the Cloud Using Machine Learning Classification

- Amélioration de l'utilisation des ressources et du temps de réponse dans le *cloud*
- Prise en compte de la charge système
- Utilisation de 6 algorithmes d'ordonnancement
- Sélection du plus performant
- Classification ML des tâches en entrée

An enhanced deadline constraint based task scheduling mechanism for cloud environment I

- Mécanisme d'ordonnancement pour les *deadlines* à risque
- Notion de baux
- Ordonner le temps alloué aux tâches et l'utilisation maximale des ressources
- L'ordonnancement se fait selon l'ordre d'arrivée
- La *deadline* est associée à une contrainte de temps
- Utilisation de l'heure de début + *deadline* comme paramètres d'ordonnancement

An enhanced deadline constraint based task scheduling mechanism for cloud environment II

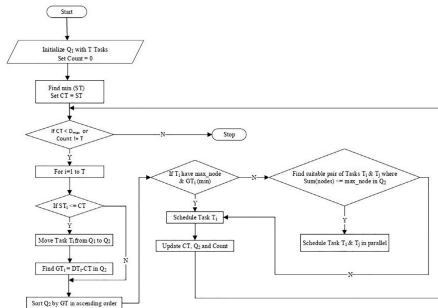


Figure 1 – *Flow chart* de la sélection des tâches

An enhanced deadline constraint based task scheduling mechanism for cloud environment III

Q_1, Q_2	Files pour stocker les baux
L	Nombre de baux à ordonnancer
L_i	N'importe quel bail dans L
CT	Temps courant
AT_{max}	Temps max d'arrivée des L baux
ST_{min}	Temps min du début des L baux
D_{max}	<i>Deadline</i> max des L baux
ST_i	Temps de début du bail L_i
GT_i	Temps restant du bail L_i
$L_i(n)$	Nombre de nœuds (VM) requis par le bail

Table 2 – Terminologie d *flow chart*

Résumé des articles I

Article	Technique	Bénéfices	Limitations
1	<i>Deep reinforcement</i>	Efficace sur l'ordonnancement des ressources <i>cloud</i> hors-ligne	Approche pourrait être étendue pour résoudre des problèmes d'ordonnancement plus compliqués que proposés
2	ANN et RL	Optimisation de l'ordonnancement en utilisant de l'ANN	Le processus d'encodage (entrée) pourrait être améliorée pour des environnements hétérogènes

Résumé des articles II

Article	Technique	Bénéfices	Limitations
3	<i>Multi-objective task scheduling</i>	Améliore le débit de DC sans perdre en <i>Service-level agreement</i>	QoS manquant dans ces travaux
4	<i>Task scheduling model</i>	Les algorithmes de TSFC ont de meilleurs performances pour l'ordonnancement des tâches	Problème de bande-passante entre les processeurs

Résumé des articles III

Article	Technique	Bénéfices	Limitations
5	<i>Supervised learning</i>	Amélioration de l'utilisation des ressources et du temps de réponse dans le <i>cloud</i>	Difficile de classer les <i>data set</i> pour minimiser les attributions lors de l'ordonnement
6	<i>First come, first serve for back-filling</i>	Mécanisme de <i>deadline</i> pour l'ordonnement des tâches	Coût pour intervertir les VM élevé

Table 3 – Résumé des articles

Consommation d'énergie

- Problème principal : utiliser moins d'énergie sans perdre en performances
- Consommation d'énergie inégale entre serveurs ← dépend des services utilisés
- Comment réaffecter les services parmi les serveurs ?

Optimisation de la consommation d'énergie par l'IA I

Exemple (Articles de recherches associés)

- ① Reinforcement Learning Based Methodology for Energy-efficient Resource Allocation in Cloud Data Centers
- ② Using Machine Learning to Reduce the Energy Wasted in Volunteer Computing Environments
- ③ Energy-aware hybrid fruitfly optimization for load balancing in cloud environments for EHR applications
- ④ Energy Efficient Task Scheduling in Cloud Environment
- ⑤ Energy efficient strategy for placement of virtual machines selected from underloaded servers in compute Cloud

Reinforcement learning based methodology for energy-efficient resource allocation in cloud data centers

- Propose un *framework* pour optimiser les allocations ressources
- Spécifiquement pour l'énergie
- 4 morceaux
 - ① *Energy-efficient Resource Allocator*
 - ② *Service Level Agreement Manager*
 - ③ *Data Center Energy Analyzer*
 - ④ *Cloud Infrastructure Resource Monitor*
- RL pour l'allocation de l'énergie

Using Machine Learning to reduce the energy wasted in Volunteer Computing Environments

- Propose une procédure pour convertir les *trace-log* des interactions des utilisateurs pour qu'ils soient compatible avec du ML
- Chaque interaction est un tuple

$$(L^i, c, L^o, i) \quad (12)$$

avec L^i et L^o les heures de *login* et *logout*, c l'ordinateur utilisé et i la durée prédite d'*idle*

- Gestion des *reboot* avec et sans utilisateur
- Gestion du côté épars des données
- Évaluation avec
 - ① *Random Forest*
 - ② *MultiLayer Perceptron*
 - ③ *Ensemble Approaches*

Energy-aware hybrid fruitfly optimization for load balancing in cloud environments for EHR applications

- Hybridation de l'algorithme *fruit fly* avec une « re-cuisson » pour trouver la solution optimale
- Équilibrage des tâches dans l'environnement *cloud*
- Définition d'une fonction multi objectifs pour équilibrer la charge des tâches sur les VM et optimiser la durée, l'énergie et le coût

Energy Efficient Task Scheduling in Cloud Environment

- Propose un algorithme d'optimisation pour l'ordonnancement des *jobs* utilisateur
- But : économie d'énergie
- Utilise un algorithme de *Clonal Selection Algorithm* pour calculer la durée totale de calcul

Energy efficient strategy for placement of virtual machines selected from underloaded servers in compute Cloud I

- Calcule d'un seuil min pour l'utilisation totale du DC
- Prise en compte de tous les hôtes actifs sur le DC
- Prédiction du nombre max d'hôtes qui peuvent être libérés selon la charge totale du DC
- Cette valeur est utilisée pour obtenir le seuil
- Les hôtes sont divisés en deux catégories
 - ① Les hôtes dont l'utilisation est en-dessous du seuil
 - ② Les hôtes dont l'utilisation est au moins égal au seuil
- Sélection des VM de la liste 1 pour essayer de les mettre dans la liste 2 avec le minimum de changement dans la consommation d'énergie

Energy efficient strategy for placement of virtual machines selected from underloaded servers in compute Cloud II

- ➊ Ajout de tous les hôtes sur-utilisés et éteints dans deux listes pour ne pas les prendre en compte
- ➋ Calcul du nombre max d'hôtes qui peuvent être libérés (H_{MAX}) selon la charge du DC

$$H_{MAX} = H_{TOTAL} - \frac{W_{TOTAL}}{UpperThreshold} \quad (13)$$

avec $W_{TOTAL} = \sum_{i=1}^{i=H_{TOTAL}} U_i$ et U_i l'utilisation du i^e hôte

- ➌ Seuil bas T_{LOWER} calculé selon H_{MAX}

Energy efficient strategy for placement of virtual machines selected from underloaded servers in compute Cloud III

- ④ Utilisation de ce seuil pour récupérer l'hôte sous-utilisé
- ⑤ Tentative de migrer toutes ses VM vers un autre hôte
- ⑥ Boucle pour tous les hs ou qu'il n'y ait plus d'hd disponible

Résumé des articles I

Article	Technique	Bénéfices	Limitations
1	RL et <i>fuzzy logic</i>	Gestion des ressources physiques en utilisant des <i>patterns</i> de demande de ressources dynamique	Ne reflète que les sources d'énergie et la consommation CPU. Pas d'influence sur l'ordonnancement
2	<i>Random forest</i> et perceptron multi-couches	Peu d' <i>overhead</i> et économie d'énergie	Besoin de données réels de la plateforme

Résumé des articles II

Article	Technique	Bénéfices	Limitations
3	<i>Multi-objective hybrid fruit fly optimization</i>	Économie d'énergie en faisant dormir les serveurs	Pourrait être étendu pour prendre en compte la QoS
4	Ordonnement des tâches par sélection de clones	Résout le problème d'ordonnement de tâches quand le nombre de DC et de jobs change dynamiquement	La répartition de la charge et le débit pourraient être ajoutés pour le rendre plus robuste

Résumé des articles III

Article	Technique	Bénéfices	Limitations
5	Algorithme prenant en compte l'utilisation de l'hôte	Étude pour calculer le seuil min qui est utilisé en sous-charge de l'hôte	Pourrait être étendu à un environnement temps réel

Table 4 – Résumé des articles

Pour aller plus loin

- Machine learning for energy-resource allocation, workflow scheduling and live migration in cloud computing :
State-of-the-art survey
 - Beaucoup d'articles sont payants
 - URCA paie des abonnements
- ⇒ Disponibles si dans le réseau de l'URCA
- ⇒ Disponibles *via* le VPN de l'URCA